Reads are open. Writes are fenced.

Animated explainer · src/sandbox.js

workspace.js answers whether a path stays inside the root. sandbox.js answers a narrower, louder question: may the agent change this file at all? Every mutation goes through one gate, and every refusal is a visible line.

9 scenes · 3 mutating actions · 1 check gate · 0 silent writes

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The contract 4

03 · The gate 5

04 · Refusals 6

05 · Shell writes 7

06 · Tokenizing 8

07 · Write targets 9

08 · The limit 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

Curiosity is allowed. Mutation is not.

SRC/SANDBOX.JS · *source*

```
export const ACTIONS = Object.freeze({
  SAVE: 'save',
  EDIT: 'edit',
  DELETE: 'delete',
});

// Every one of them goes through check().
// A refusal prints one loud FORBIDDEN line.
```

Reads — Anywhere. A read that leaves the sandbox is a bad idea, not an incident.

Writes — Only inside the workspace. This is the incident boundary.

> **remember** The fence is around mutations, not curiosity. That is the whole design.
> **On screen:** workspace.js resolves paths. sandbox.js decides which actions are allowed to mutate. Reads can go anywhere, because an agent that cannot read docs or logs is useless. Writes, edits and deletes are confined. That is where damage outlives the run.

02 · The contract

Three actions. One answer.

save — Create or overwrite a file inside the root.

edit — Replace a snippet in an existing file.

delete — Remove a file or an empty folder.

> The root itself is not a file. Deleting it would take the sandbox with it, so that is refused too.
> **On screen:** The sandbox governs exactly three actions: save, edit and delete. Reads are handled by workspace.js. These three are the mutations. One gate answers all three, so “was this allowed?” has exactly one place to look.

03 · The gate

One gate. Every write.

SRC/SANDBOX.JS · *source*

```
check(action, requested) {
  if (!ALLOWED.has(action)) {
    throw new TypeError('unknown sandbox action "' + action + '"');
  }

  let target;
  try {
    target = this.workspace.resolve(requested);
  } catch (err) {
    if (err instanceof PathError) this.forbid(action, requested, err.message);
    throw err;
  }

  if (action === ACTIONS.DELETE && target.rel === '.') {
    this.forbid(action, requested, 'the sandbox root itself cannot be deleted');
  }

  return target;
}
```

The path policy lives in workspace.js. This module never re-implements it; it only decides which actions are sandboxed.

On screen: check is the single gate. It rejects unknown actions first. Then it asks workspace.resolve to enforce containment. A PathError becomes a loud refusal. Every successful write gets an absolute path and a relative label.

04 · Refusals

A refusal is a line, not a stack trace.

SRC/SANDBOX.JS · *source*

```
forbid(action, requested, reason) {
  this.refusals += 1;
  this.log(ui.forbidden(action, requested, reason));
  throw new ForbiddenError(action, requested, reason);
}
```

THE CONSOLE · *text*

```
⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

> The error message has the same shape as the console line, so the model and the human read the same sentence.
>
> **On screen:** A refusal prints one bold red line and throws a ForbiddenError. The error carries the action, the requested path and the reason. The model receives it as a tool result, not as a crash, and can try another path.

05 · Shell writes

A command is not a way round the fence.

- > Redirections the destination after > or >>.
- 📋 Copy and move the last positional destination.
- 🗑️ Delete commands every positional operand.
- 🌐 Downloaders the output flag for curl and wget.
- 🪟 PowerShell Out-File, Set-Content, New-Item and friends.

> This reads the command line, not the code inside a script. It is best effort, and the module says so.
> **On screen:** run_command is a real shell. Its obvious write targets are read from the command line. Redirections, copy destinations, delete operands, downloader output flags and PowerShell cmdlets are checked. This is not a shell sandbox. A script can still write anywhere; a container is the answer to that.

06 · Tokenizing

Split the line honestly.

SRC/SANDBOX.JS · *source*

```
for (let i = 0; i < text.length; i += 1) {
  const ch = text[i];
  if (quote) {
    if (ch === quote) quote = null;
    else current += ch;
    continue;
  }
  if (ch === '"' || ch === "'") { quote = ch; continue; }
  if (/\s/.test(ch)) { flush(); continue; }
  if (ch === '>' || ch === '<' || ch === '|' || ch === '&' || ch === ';') {
    flush();
    // ...emit one or two-character operator...
  }
  current += ch;
}
```

> Quotes are removed so a path with spaces stays one word. Operators become their own tokens.
> **On screen:** The tokenizer respects quotes and separates shell operators. A quoted path with spaces remains one token, which is what the write check needs. It is not a full shell parser. It is a small, honest attempt at the obvious cases.

07 · Write targets

Who writes, and where.

SRC/SANDBOX.JS · *source*

```
if (ALL_POSITIONAL.has(verb)) {
  for (const target of positional) targets.push({ verb, target });
  return;
}
if (DEST_LAST.has(verb)) {
  if (positional.length >= 2) targets.push({ verb, target: positional[positional.length - 1] });
  return;
}
if (verb === 'robocopy') {
  if (positional.length >= 2) targets.push({ verb, target: positional[1] });
  return;
}
```

DEST_LAST — copy, cp, move, mv, xcopy, scp, rsync.

ALL_POSITIONAL — del, rm, rmdir, mkdir, touch, tee.

OUTPUT_FLAG — curl -o, wget -O.

> **On screen:** The command name decides how to find its write targets. For copy-like commands it is the last positional. For delete-like commands it is every positional. Downloaders and PowerShell cmdlets get their own rules. The result is one list of paths to check.

08 · The limit

Honest about what it cannot stop.

- ✅ Catches the obvious write targets named on the command line.
- 🚫 Does not catch node -e, a downloaded script, or a program that writes internally.
- 📦 Real answer a container or OS-level sandbox.

> **remember** A guard that pretends to be complete is worse than a guard that names its limits.
> **On screen:** The shell check is best effort, not a sandbox for the shell. A script can still call fs.copyFileSync or download another program. The module says so plainly. A container is the answer when the threat model needs one.

09 · Recap

One gate. Loud refusals.

• 3 mutating actions

• 1 check gate

• 0 silent writes

→

Reads are open. Curiosity is not the incident.

→

Writes are fenced. Save, edit and delete go through check().

→

Refuse loudly. A FORBIDDEN line reaches the model and the human.

→

Name the limits. The shell check is best effort, not a container.

> **remember** A sandbox is not a folder. It is one gate that every mutation must pass.
> **On screen:** Three mutating actions, one gate, no silent writes. Reads stay open, writes go through check, and refusals are loud and explainable. That is the difference between a workspace and a sandbox.